Chapter 17 | Exercise solutions with Python

Designing Reproducible Evaluations

This notebook solves Exercises 5–8. It checks that grouped splits keep related rows together, separates empty calibration bins from observed rates, compares paired bootstrap units, and audits a prediction join. Exercises 1–4 and 9–11 are in the written solutions.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch17_evaluation"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Check a split by group

Suppose each row is a message and its group ID identifies the customer who sent it. Customer 1 sent the first two messages. If those messages go into different splits, the model can learn customer-specific patterns in training and encounter them again during evaluation. To evaluate on held-out customers, assign each customer to one split and send all of that customer's rows there.

The eight rows have group IDs $(1, 1, 2, 3, 3, 4, 5, 5)$, so there are five distinct groups. Give `split_by_group` the group counts $(3, 1, 1)$ for train, validation, and test, with seed 7. The helper shuffles the distinct IDs and takes the first three for train, the next one for validation, and the last one for test. It then finds the original row indices for the IDs in each split.

```
from examples.ch17_evaluation.model import split_by_group

split_group_ids = np.array([1, 1, 2, 3, 3, 4, 5, 5])
split_group_counts = (3, 1, 1)
split_seed = 7
split_rows = split_by_group(
    split_group_ids, split_group_counts, seed=split_seed
)

split_names = ("Train", "Validation", "Test")
split_groups = [
    set(split_group_ids[rows].tolist()) for rows in split_rows
]
shuffled_ids = np.random.default_rng(split_seed).permutation(
    np.unique(split_group_ids)
)
print("Seeded group order", shuffled_ids.tolist())
for name, rows, groups in zip(split_names, split_rows, split_groups):
    print(f"{name:<10} rows {rows.tolist()}, groups {sorted(groups)}")
    print(f"{'':10} {len(rows)} row(s), {len(groups)} group(s)")

train_groups, validation_groups, test_groups = split_groups
assert train_groups.isdisjoint(validation_groups)
assert train_groups.isdisjoint(test_groups)
assert validation_groups.isdisjoint(test_groups)
assert tuple(map(len, split_groups)) == split_group_counts
assert tuple(map(len, split_rows)) == (6, 1, 1)
np.testing.assert_array_equal(
    np.sort(np.concatenate(split_rows)),
    np.arange(len(split_group_ids)),
)
print("The groups are disjoint and every row appears once.")
```

OUTPUT

```
Seeded group order [3, 1, 5, 2, 4]
Train      rows [0, 1, 3, 4, 6, 7], groups [1, 3, 5]
           6 row(s), 3 group(s)
Validation rows [2], groups [2]
           1 row(s), 1 group(s)
Test       rows [5], groups [4]
           1 row(s), 1 group(s)
The groups are disjoint and every row appears once.
```

Train gets groups $\{1, 3, 5\}$, validation gets $\{2\}$, and test gets $\{4\}$. The row indices start at zero. In train, rows 0 and 1 belong to group 1, rows 3 and 4 to group 3, and rows 6 and 7 to group 5. The seeded order sets membership. The returned rows appear in their original order.

Each of the three train groups supplies two rows, giving six training rows. The validation and test groups each supply one row. Thus the split has group counts $(3, 1, 1)$ and row counts $(6, 1, 1)$. A quota in groups gives a row count that depends on the sizes of the selected groups.

6. Inspect an empty bin

Imagine four messages scored for whether they are updates. The scores 0 and 0.1 belong to messages that are lookups (label 0). The scores 0.9 and 1 belong to updates (label 1). We want to compare the scores with how often updates actually occur among messages with similar scores.

Use edges $(0, 0.25, 0.75, 1)$. Each bin includes its left edge and excludes its right edge, with one exception. The last bin also includes 1, so every valid probability belongs to a bin. A score exactly equal to 0.25 goes in the middle bin, and a score exactly equal to 0.75 goes in the last bin.

```
from examples.ch17_evaluation.model import reliability_bins

bin_labels = np.array([0, 0, 1, 1])
bin_scores = np.array([0.0, 0.1, 0.9, 1.0])
bin_edges = np.array([0.0, 0.25, 0.75, 1.0])
counts, mean_scores, positive_rates, brier = reliability_bins(
    bin_labels, bin_scores, bin_edges
)
intervals = ("[0, 0.25)", "[0.25, 0.75)", "[0.75, 1]")

print(f"{'Bin':<13} {'Count':>5} {'Mean score':>12} {'Positive rate':>15}")
for interval, count, mean, rate in zip(
    intervals, counts, mean_scores, positive_rates
):
    print(f"{interval:<13} {count:>5} {mean:>12.2f} {rate:>15.2f}")
print(f"Brier score {brier:.3f}")

np.testing.assert_array_equal(counts, [2, 0, 2])
np.testing.assert_allclose(mean_scores[[0, 2]], [0.05, 0.95])
np.testing.assert_allclose(positive_rates[[0, 2]], [0.0, 1.0])
assert np.isnan(mean_scores[1]) and np.isnan(positive_rates[1])
np.testing.assert_allclose(brier, 0.005)
```

```
OUTPUT
Bin           Count   Mean score   Positive rate
[0, 0.25)         2         0.05            0.00
[0.25, 0.75)      0          nan             nan
[0.75, 1]         2         0.95            1.00
Brier score 0.005
```

A reliability plot puts the mean predicted probability on the horizontal axis and the fraction of positive labels on the vertical axis. Use the bins whose counts are positive.

Bin	Scores	Mean probability	Positive fraction	Plot point
$[0, 0.25)$	$0, 0.1$	$(0 + 0.1)/2 = 0.05$	$0/2 = 0$	$(0.05, 0)$
$[0.25, 0.75)$	None	Undefined	Undefined	Omit
$[0.75, 1]$	$0.9, 1$	$(0.9 + 1)/2 = 0.95$	$2/2 = 1$	$(0.95, 1)$

The low bin is 0.05 below the ideal diagonal $y = x$ because the mean score exceeds the observed fraction. The high bin is 0.05 above it because the observed fraction exceeds the mean score. Each point summarizes two examples.

In the middle bin, both averages would require dividing by zero. The helper returns `NaN` for both coordinates to mark them as undefined. Filling both with zero would draw a fake perfectly calibrated point at $(0, 0)$. Plotting a zero rate at the bin midpoint, $(0.50, 0)$, would instead suggest that predictions near 0.50 all had negative outcomes. The data contain no examples in that range. The low bin's zero rate is a measured $0/2$, whereas the middle bin's count of zero means its rate is undefined.

The helper also returns the Brier score. It averages the squared error between each original score and its label. Here the errors are 0, 0.01, 0.01, and 0, so the Brier score is $0.02/4 = 0.005$.

7. Change the resampling unit

Models A and B label the same six messages. The first customer sent two messages, and B gets both right where A gets both wrong. The last customer sent one message that only A gets right and one that both models get right. The table keeps the two predictions together for each message.

Message	Customer	True label	A	B	Correctness difference
1	0	1	0	1	+1
2	0	1	0	1	+1
3	1	0	0	0	0
4	1	0	1	1	0
5	2	1	1	0	−1
6	2	0	0	0	0

On message i, the correctness difference d_i is 1 if only B is right, -1 if only A is right, and 0 if both have the same outcome. The difference in accuracy is the average of these row differences. A gets three messages right and B gets four, so the observed gap is

$$\Delta = \frac{4}{6} - \frac{3}{6} = \frac{1 + 1 + 0 + 0 - 1 + 0}{6} = \frac{1}{6}.$$

A paired bootstrap repeatedly draws a new sample, allowing repeats, from the observed test set. A row draw keeps A, B, and the true label together. For example, drawing messages 1, 1, 2, 4, 5, and 6 gives a gap of $(1 + 1 + 1 + 0 - 1 + 0)/6 = 1/3$ for that draw.

With customer resampling, each draw selects three customers and brings along every message from each selected customer. For example, drawing customers 0, 0, and 2 contributes two copies of customer 0's gain of two correct messages and one copy of customer 2's loss of one. The gap is $(2 + 2 - 1)/6 = 1/2$. The denominator counts all sampled messages, including repeats. This gives each sampled message equal weight even if customers have different message counts.

```
from examples.ch17_evaluation.model import paired_accuracy_bootstrap

bootstrap_truth = np.array([1, 1, 0, 0, 1, 0])
bootstrap_a = np.array([0, 0, 0, 1, 1, 0])
bootstrap_b = np.array([1, 1, 0, 1, 0, 0])
customer_ids = np.array([0, 0, 1, 1, 2, 2])

correct_a = bootstrap_a == bootstrap_truth
correct_b = bootstrap_b == bootstrap_truth
differences = correct_b.astype(int) - correct_a.astype(int)
observed_gap = differences.mean()
print(f"Correct messages: A {correct_a.sum()}, B {correct_b.sum()}")
print(f"B minus A by message: {differences.tolist()}")
print(f"Observed accuracy gap: {observed_gap:+.6f}")
```

```
OUTPUT
Correct messages: A 3, B 4
B minus A by message: [1, 1, 0, 0, -1, 0]
Observed accuracy gap: +0.166667
```

Compute 4,000 resampled gaps for each method with seed 5. The 95% percentile interval uses the 2.5th and 97.5th percentiles of those gaps. Both calls report the same observed gap because they score the same original six messages.

```
for unit, groups in (("Message", None), ("Customer", customer_ids)):
    point, (low, high) = paired_accuracy_bootstrap(
        bootstrap_truth, bootstrap_a, bootstrap_b,
        repeats=4000, confidence=0.95, seed=5, groups=groups,
    )
    assert np.isclose(point, observed_gap)
    print(f"{unit:<8} gap {point:+.6f}   95% [{low:+.6f}, {high:+.6f}]")
```

```
OUTPUT
Message  gap +0.166667   95% [-0.333333, +0.666667]
Customer gap +0.166667   95% [-0.500000, +1.000000]
```

Both methods estimate a gain of 16.7 percentage points for B on these six messages. The interval widens from $[-1/3, 2/3]$ for messages to $[-1/2, 1]$ for customers. With customer resampling, the first customer's two gains always travel together. A draw with three copies of that customer has a gap of $+1$. A draw with three copies of the last customer has a gap of $-1/2$.

Use the customer bootstrap when messages from a customer can be dependent and customers are the independent units. For example, two messages may use the same wording or discuss the same request. Drawing customers preserves these connections. Both intervals span zero and allow sizable changes in either direction. Six messages from just three customers give little information about the gap for new customers. These intervals describe sensitivity to the evaluation sample for the two fixed checkpoints. To study variation across training runs, compare independently trained checkpoints.

8. Audit a prediction join (optional)

Suppose the reference file has one row per message and the prediction file contains the same six messages in a different order. For `m01`, the correct label is `1` and the model's prediction is also `1`. In the shuffled file the first row belongs to `m04` and predicts `0`. Scoring by position treats that `0` as a mistake on `m01`.

An example ID is a stable key for a message across both files. Keep the IDs attached while shuffling the prediction records. First score the values in their current order to see what happens when the keys are ignored.

```
from examples.ch17_evaluation.model import binary_metrics

truth_records = [
    ("m01", 1), ("m02", 0), ("m03", 1),
    ("m04", 0), ("m05", 1), ("m06", 0),
]
predictions_in_truth_order = [
    ("m01", 1), ("m02", 1), ("m03", 1),
    ("m04", 0), ("m05", 0), ("m06", 0),
]
shuffle_order = [3, 0, 5, 2, 1, 4]
prediction_records = [
    predictions_in_truth_order[i] for i in shuffle_order
]

print("Position  Label ID  Label  Prediction ID  Prediction")
for position, (label, pred) in enumerate(
    zip(truth_records, prediction_records), start=1
):
    label_id, label_value = label
    pred_id, pred_value = pred
    print(f"{position:>8}  {label_id:>8}  {label_value:>5}  "
          f"{pred_id:>13}  {pred_value:>10}")

truth_by_position = np.array([label for _, label in truth_records])
predictions_by_position = np.array([p for _, p in prediction_records])
order_score = binary_metrics(truth_by_position, predictions_by_position)
print(f"Accuracy by file position: {order_score['accuracy']:.2%}")
```

```
OUTPUT
Position  Label ID  Label  Prediction ID  Prediction
       1       m01      1            m04           0
       2       m02      0            m01           1
       3       m03      1            m06           0
       4       m04      0            m03           1
       5       m05      1            m02           1
       6       m06      0            m05           0
Accuracy by file position: 33.33%
```

Both arrays have six values, so `binary_metrics` can compute an accuracy even when the rows represent different messages. Before scoring, check for a one-to-one match. Build an index for each file, rejecting repeated IDs before a dictionary assignment can overwrite a row. Then compare the key sets in both directions. A reference key absent from predictions is missing. A prediction key absent from the reference is unexpected. Once the keys agree, look up every prediction in the order of the reference IDs.

```
def unique_by_id(records, source):
    """Index records with one value per nonblank string ID."""
    indexed = {}
    for example_id, value in records:
        if not isinstance(example_id, str) or not example_id.strip():
            raise ValueError(f"{source} has a blank or non-string ID")
        if example_id in indexed:
            raise ValueError(f"{source} has duplicate ID {example_id!r}")
        indexed[example_id] = value
    return indexed


def align_by_id(labels, predictions):
    truth_by_id = unique_by_id(labels, "Label file")
    prediction_by_id = unique_by_id(predictions, "Prediction file")
    missing = sorted(truth_by_id.keys() - prediction_by_id.keys())
    unexpected = sorted(prediction_by_id.keys() - truth_by_id.keys())
    if missing or unexpected:
        raise ValueError(
            f"Missing IDs: {missing}. Unexpected IDs: {unexpected}."
        )
    return (
        np.array(list(truth_by_id.values())),
        np.array([prediction_by_id[key] for key in truth_by_id]),
    )
```

```
joined_truth, joined_predictions = align_by_id(
    truth_records, prediction_records
)
joined_score = binary_metrics(joined_truth, joined_predictions)
correct_ids = [
    example_id
    for (example_id, _), is_correct in zip(
        truth_records, joined_truth == joined_predictions
    )
    if is_correct
]
assert correct_ids == ["m01", "m03", "m04", "m06"]
assert np.isclose(order_score["accuracy"], 2 / 6)
assert np.isclose(joined_score["accuracy"], 4 / 6)
print(f"Correct IDs after joining: {', '.join(correct_ids)}")
print(f"Accuracy after joining: {joined_score['accuracy']:.2%}")
```

OUTPUT

```
Correct IDs after joining: m01, m03, m04, m06
Accuracy after joining: 66.67%
```

The row-order calculation reports two correct answers out of six. The ID join recovers four correct answers out of six. Try three broken prediction files to check that they are rejected before scoring. A replacement with an unexpected ID or a duplicate can leave the file with all six rows, so checking the count alone misses both errors.

```
broken_files = (
    ("Missing", prediction_records[:-1], "Missing IDs: ['m05']"),
    ("Wrong key", prediction_records[:-1] + [("m99", 0)],
     "Unexpected IDs: ['m99']"),
    ("Duplicate", prediction_records[:-1] + prediction_records[:1],
     "Prediction file has duplicate ID 'm04'"),
)
for case, records, expected in broken_files:
    try:
        align_by_id(truth_records, records)
    except ValueError as error:
        assert expected in str(error)
        print(f"{case}: {error}")
    else:
        raise AssertionError(f"{case} should fail before scoring")
```

```
OUTPUT
Missing: Missing IDs: ['m05']. Unexpected IDs: [].
Wrong key: Missing IDs: ['m05']. Unexpected IDs: ['m99'].
Duplicate: Prediction file has duplicate ID 'm04'
```

This check also rejects duplicate keys in the reference file. For a paired model comparison, run the same check for each model's file against the same reference order before passing the two arrays to the bootstrap helper.